Information and Software Technology 164 (2023) 107304

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Can an old fashioned feature extraction and a light-weight model improve vulnerability type identification performance?

Hieu Dinh Vo, Son Nguyen *

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Viet Nam

ARTICLE INFO

Keywords:
Vulnerability type identification
Vulnerability resolution
Software vulnerability

ABSTRACT

Recent advances in automated vulnerability detection have achieved potential results in helping developers determine vulnerable components. However, after detecting vulnerabilities, investigating to fix vulnerable code is a non-trivial task. In fact, the types of vulnerability, such as *buffer overflow* or *memory corruption*, could help developers quickly understand the nature of the weaknesses and localize vulnerabilities for security analysis. In this work, we investigate the problem of vulnerability type identification (VTI). The problem is modeled as the multi-label classification task, which could be effectively addressed by "*pre-training, then fine-tuning*" framework with deep pre-trained embedding models. We evaluate the performance of the well-known and advanced pre-trained models for VTI on a large set of vulnerabilities. Surprisingly, their performance is not much better than that of the classical baseline approach with an old-fashioned bag-of-word, TF-IDF. Meanwhile, these deep neural network approaches cost much more resources and require GPU. We also introduce a lightweight independent component to refine the predictions of the baseline approach. Our idea is that the types of vulnerabilities could strongly correlate to certain code tokens (distinguishing tokens) in several crucial parts of programs. The distinguishing tokens for each vulnerability type are statistically identified based on their prevalence in the type versus the others. Our results show that the baseline approach enhanced by our component can outperform the state-of-the-art deep pre-trained approaches while retaining very high efficiency. Furthermore, the proposed component could also improve the neural network approaches by up to 92.8% in macro-average F1.

1. Introduction

Software vulnerabilities are weaknesses in a software system that could be exploited by attackers. This exploitation can cause substantial damage, especially for the critical systems [1]. To reduce manual effort in discovering vulnerabilities, researchers have invested considerable effort in investigating effective approaches for *automated vulnerability detection*, leading to many techniques [2–12]. Recent advances in this field have resulted in a (quite) high accuracy in determining whether an entire given method/file is vulnerable or not [3,5–8]. For example, with *BigVul* benchmark [13], Fu et al. [3] propose *LineVul*, which achieves more than 90% accuracy in vulnerability detection at the function level. With the support of these state-of-the-art techniques, the next step, security analysis, that developers have to perform is investigating the detected vulnerable functions to determine the actual presence of the vulnerability. However, even having vulnerable code, investigating to fix those functions could still be a non-trivial task [14–16].

Meanwhile, the knowledge of vulnerability types, such as *buffer overflow* or *memory corruption*, can offer significant insights to developers when debugging the vulnerable code. This information serves as a guide to understanding the principles behind the vulnerability, enabling developers to swiftly pinpoint the exact location of the vulnerability and propose potential approaches to rectify the flawed code [17]. For instance, if a developer identifies a vulnerable function, as shown in Fig. 1, and knows it contains a buffer overflow error, she can immediately focus her investigation on the code statements responsible for writing or copying data into the buffer. This allows her to assess the likelihood of these statements exceeding the buffer's boundaries and overwriting adjacent memory locations. Consequently, she can initiate security analysis by examining the code statements at lines 7, 19, and 30 instead of analyzing the entire function. Once the vulnerable statement (line 30) is identified, she can employ established techniques, such as implementing size checks before writing or correcting the amount of data to be copied, to mitigate the vulnerability. Thus, by determining the vulnerability types after detecting vulnerable code,

* Corresponding author.
E-mail addresses: hieuvd@vnu.edu.vn (H.D. Vo), sonnguyen@vnu.edu.vn (S. Nguyen).

https://doi.org/10.1016/j.infsof.2023.107304
Received 16 January 2023; Received in revised form 26 June 2023; Accepted 24 July 2023
Available online 28 July 2023
0950-5849/© 2023 Elsevier B.V. All rights reserved.

```
static long ec_device_ioctl_xcmd(struct cros_ec_dev *ec,
     void __user *arg)
{
   long ret;
   struct cros_ec_command u_cmd;
   struct cros_ec_command *s_cmd;

   if (copy_from_user(&u_cmd, arg, sizeof(u_cmd)))
      return -EFAULT;

   if ((u_cmd.outsize > EC_MAX_MSG_BYTES) ||
       (u_cmd.insize > EC_MAX_MSG_BYTES))
      return -EINVAL;

   s_cmd = kmalloc(sizeof(*s_cmd) + max(u_cmd.outsize,
         u_cmd.insize),
         GFP_KERNEL);
   if (!s_cmd)
      return -ENOMEM;

   if (copy_from_user(s_cmd, arg, sizeof(s_cmd) +
         u_cmd.outsize)) {
      ret = -EFAULT;
      goto exit;
   }

   s_cmd->command += ec->cmd_offset;
   ret = cros_ec_cmd_xfer(ec->ec_dev, s_cmd);
   /* Only copy data to userland if data was received. */
   if (ret < 0)
      goto exit;

   if (copy_to_user(arg, s_cmd, sizeof(*s_cmd) +
         u_cmd.insize))
      ret = -EFAULT;
exit:
   kfree(s_cmd);
   return ret;
}
```

Fig. 1. A *buffer overflow* in Linux Kernel, CVE-2016-6156.

developers and code auditors can significantly reduce their workload, which is especially beneficial when dealing with large sections of vulnerable code.

Despite the importance of vulnerability type identification (VTI) in debugging vulnerable code after being detected, the problem has not received the deserved attention. In this work, we make the first step to explore the problem of VTI by using the existing techniques in software engineering (SE) and natural language processing (NLP). Particularly, we model VTI as the multi-label text classification task in NLP [18–20]. This is reasonable because each vulnerable function f could be considered as a document d, and the type set of the vulnerabilities in f could be the label set of d.

The NLP community recently witnessed a dramatic paradigm shift towards the "*pre-training + fine-tuning*" framework. Deep pre-trained models, e.g., BERT [21], induce powerful embeddings that can be rapidly fine-tuned on many downstream NLP problems by adding a task-specific lightweight linear layer on top of the transformer models. BERT-like models (e.g., XLNet [22] and RoBERTa [23]) have led to state-of-the-art performance on many NLP tasks, such as part-of-speech tagging or text classification. For the SE, pre-trained models have recently shown to be highly effective in many classification tasks such as bug detection, clone detection, and vulnerability detection [2,3,24–27]. This naturally raises a question: *How are these pre-trained models effective for the VTI task?*

Additionally, we suspect that given a set of all possible vulnerability types T and a vulnerable function f, the types of the vulnerability in f, $S \subseteq T$ could be determined by the appearance of code tokens in f. For example, *directory traversal* vulnerabilities, which allow attackers to access files/directories stored outside the web root directory, usually contain certain code (sub)tokens such as `file`, `base_path`, or `directory`. Another example is that *buffer overflow* errors usually associate with `buffer` (`buf`) or `copy` (`cpy`). Meanwhile, the importance or relevance of these (sub)tokens in determining the types of vulnerabilities could be captured well by the old-fashioned code representations such as TF-IDF features. Another natural question is: *How does a simple classification model with an old-fashioned code representation such as TF-IDF work for the VTI task compared to pre-trained models?*

In this paper, we conduct experiments to evaluate the performance of the state-of-the-art methods in natural language processing (NLP) and software engineering (SE) for the VTI task. We select two pre-trained models, Word2vec [28] and CodeBERT [29], which are the representative non-contextual and contextual code embeddings [30]. We pick Word2vec because this model has become one of the most popular pre-trained models for code due to its efficiency [24,25]. Meanwhile, the reasons for our selection of CodeBERT are the model's reputation and its strong improvements in many SE tasks [27]. We also compare these state-of-the-art models against a simple multi-label classification model, Binary Relevance (BR), with old-fashion TF-IDF features. With the BR classifier and TF-IDF, this approach is considered as the baseline of multi-label text classification in NLP [20] (so-called Base).

Our experiments on BigVul benchmark [13] show a *surprising result* that *the advanced pre-trained models just slightly improve the VTI performance of the baseline approach by only less than 7% in classification accuracy*. Meanwhile, the deep pre-trained models require GPUs and cost up to 40X and 5X more time in training and predicting compared to the baseline approach. These show that for the VTI task, TF-IDF and classical Binary Relevance could capture well the features to distinguish vulnerability types and *efficiently* achieve performance very competitive with the deep pre-trained models in VTI.

In this work, we also introduce a simple technique to improve the VTI performance of Base. Our idea is that certain code (sub)tokens are more likely to appear/not appear in the vulnerabilities of a type than the others. These *distinguishing tokens* are identified beforehand (before predicting) by statistically analyzing the syntactic code elements crucial for VTI (e.g., *function calls*, *assignments*, or *control structures*) in previously known vulnerable code. As an *independent component*, these distinguishing tokens are used to refine the predictions produced by Base. The intuition is that if a function is predicted to contain a vulnerability *not* of type t, yet actually contains the distinguishing tokens of t which are prevalent in the cases of t but not the other types; then the prediction will be refined to the vulnerability of t.

Our experimental results show that the combination of Base and our component significantly improves the VTI performance of Base and outperforms the advanced pre-trained VTI models. Meanwhile, the predicting time slightly increases, and identifying distinguishing tokens does not increase the overall training and preparing time. These results indicate that Base *combining with a very lightweight component could improve the VTI performance of advanced pre-trained models while retaining a very high efficiency*. We also show that our technique could effectively improve the other VTI approaches when applied on top of them up to 92.8% F1-score.

In brief, this paper makes the following contributions:

1. An exploratory study on the performance of both traditional and advanced VTI techniques.
2. Surprising experimental results showing that a simple model with an old-fashioned feature extraction could achieve a very competitive performance with the state-of-the-art approaches.
3. A lightweight but effective technique improving the performance of the existing approaches.

The detailed implementation and dataset can be found at: *https://github.com/sonnguyenvnu/VIT-Project*.

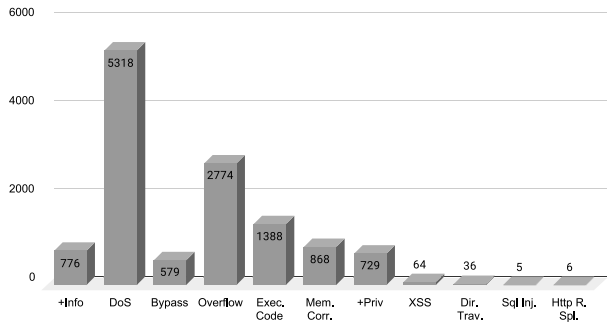

Fig. 2. The number of cases by vulnerability types.

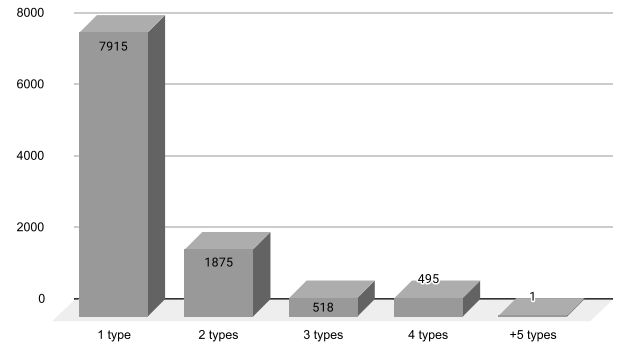

Fig. 3. The number of cases by the number of types.

The rest of this paper is organized as follows. Section 2 states the problem of vulnerability type identification (VTI) with a benchmark and evaluation metrics for VTI approaches. Then, several VTI approaches modeling the VTI task as the multi-label classification task and our surprising results are introduced in Section 3. After that, Section 4 introduces the design of our lightweight independent technique and its effectiveness in improving Base as well as the other approaches. Some threats to validity are discussed in Section 5. Section 6 provides the related work. Finally, Section 7 concludes this paper.

2. Vulnerability type identification

Given a finite set of vulnerability types T and a vulnerable function f, the vulnerability type identification (VTI) task associates a subset of types $S \subseteq T$ with the function f. A vulnerable function dataset D consists of N vulnerable functions with their vulnerability types (f_1, S_1), (f_2, S_2), $\ldots$, (f_N, S_N). In this work, *we model the problem of vulnerability type identification as multi-label text classification* [18]. This is reasonable because each vulnerable function could be considered as a document, and the types of vulnerabilities in the function could be considered as the label/tag set of the document. Note that each function in D is a vulnerable one which could be effectively detected by the existing vulnerability detection techniques [2–8].

2.1. Dataset

Data selecting. To evaluate VTI approaches, we use BigVul, which is one of the largest vulnerability datasets and provides the vulnerability types of each case. The dataset is collected from 348 real-world C/C++ projects on GitHub, such as Chromium, Linux, Android, PHP, OpenSSL, QEMU, and FFmpeg. The dataset includes about 10,900 vulnerable functions in 13 vulnerability types and 44,603 vulnerable lines of code. In this dataset, the types of each vulnerable function are extracted from its corresponding CVE-Details.

Figs. 2 and 3 show the statistical information of BigVul on vulnerability types and the number of types of vulnerability in each function. Particularly, *Denial of Service (DoS)* and *Overflow* are the two most popular types of vulnerability in the dataset. This is expected because *Denial of Service (DoS)* and *Overflow* are two of the most frequent vulnerability types [31]. Meanwhile, *Sql Injection (Sql Inj.)* and *Http Response Splitting (Http R.Spl)* are very rare in BigVul. As seen in Fig. 3, the vulnerabilities in most of the functions belong to only one type (about 72%). Additionally, there are more than 1 K vulnerable functions having more than two types.

There are some other public vulnerability datasets, but they are not suitable for being used in our experiments. Particularly, SATE IV Juliet [32] is a synthetic vulnerability dataset that is manually created from known vulnerable patterns. However, Chakraborty et al. [11] have demonstrated that this dataset contains only simple vulnerabilities which do not reflect real-world vulnerabilities. Meanwhile, Devign [10] and Reveal [11], which are constructed from the vulnerabilities in real-world open-source projects, do not provide the vulnerability types for the vulnerable functions. Thus, we do not use these vulnerability datasets in our evaluation experiments.

Data splitting. To evaluate VTI methods, we use a random training/validation/test split ratio of 80:10:10, i.e., the whole BigVul is split into 80% of training data, 10% of validating/fine-tuning data, and 10% of testing data.

2.2. Evaluation metrics

To evaluate VTI approaches, we apply the evaluation metrics which are widely used in multi-label classification studies [19]. For each function f_i in the test set, the types of f_i is represented by a vector Y_i such that for $j \in [1, \|T\|]$, $Y_{ij} = 1$ if $T_j \in T$ is one of the vulnerability types of f, otherwise $Y_{ij} = 0$. Let denote vector Z_i the vulnerability type prediction for f_i produced by a VTI technique. For a test set containing N cases (functions), the considering evaluation metrics including *exact match ratio*, *hamming score*, *accuracy*, *macro-average*, *micro-average*, *weighted-average*, *sample-average*, are computed as follows.

Exact match ratio indicates the percentage of vulnerable functions (cases) that have *all labels* predicted correctly:

$$ExactMatchRatio = \frac{1}{N}\sum_{i=1}^{N}\mu(Y_i = Z_i)$$

where μ returns 1 if Y_i and Z_i are exactly matched, $\forall j \in [1, \|T\|], Y_{ij} = Z_{ij}$, and 0 otherwise. *Exact match ratio* is the most strict one among the metrics. We also use other less strict metrics for the multi-labels classification task.

Hamming score is defined as the proportion of the correctly predicted types to the total number of predicted types and actual types for each case. The overall hamming score is the average across all cases.

$$HammingScore = \frac{1}{N}\sum_{i=1}^{N}\frac{\|Y_i \cap Z_i\|}{\|Y_i \cup Z_i\|}$$

Note that $\|Y_i \cap Z_i\| = \sum_{j=1}^{\|T\|}(Y_{ij} \wedge Z_{ij})$ and $\|Y_i \cup Z_i\| = \sum_{j=1}^{\|T\|}(Y_{ij} \vee Z_{ij})$. As seen, *Hamming score* only considers label 1 in Y_i and Z_i. We additionally use *accuracy*, which considers matching for both labels 1 and 0.

In this work, *accuracy* is calculated as the following formula:

$$Accuracy = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\|T\|}\sum_{j=1}^{\|T\|}\mu(Y_{ij} = Z_{ij})$$

There are several different methods to measure a multi-label classifier by averaging out the types: *micro-averaging*, *macro-average*, *weighted-average*, and *sample-average*. For *micro-averaging*, all true-positive cases (TPs), true-negative cases (TNs), false-positive cases (FPs), and false-negative cases (FNs) for each type are summed up, and then the average

is taken. In the micro-averaging method, we sum up the individual TPs, FPs, and FNs of the system for different sets and then apply them.

$$Precision^{micro} = \frac{\sum_{t \in T} TPs(t)}{\sum_{t \in T} TPs(t) + FPs(t)}$$

$$Recall^{micro} = \frac{\sum_{t \in T} TPs(t)}{\sum_{t \in T} TPs(t) + FNs(t)}$$

And the *micro-average* F1-score will be simply the harmonic mean of the above two equations.

$$F1^{micro} = \frac{2 \times Precision^{micro} \times Recall^{micro}}{Precision^{micro} + Recall^{micro}}$$

Macro-average is straight forward. We just take the average of the precision and recall of the system on different sets.

$$Precision^{macro} = \frac{\sum_{t \in T} Precision(t)}{\|T\|}$$

$$Recall^{macro} = \frac{\sum_{t \in T} Recall(t)}{\|T\|}$$

$$F1^{macro} = \frac{2 \times Precision^{macro} \times Recall^{macro}}{Precision^{macro} + Recall^{macro}}$$

Weighted-average is simply the average of the precision and recall for individual classes weighted by the support of that class. Meanwhile, to compute *Sample average*, precision, recall, and F1-score are computed for each case and then averaged them.

3. Vulnerability type identification as multi-label classification

In general, multi-label classification could be addressed by two main approaches: *problem transformation methods* and *adapted methods*. *Problem transformation methods* transform a multi-label problem into multiple binary classification problems. In this fashion, the binary classifier of each label $l \in L$ can be employed to make the classifications, and these are then transformed back into multi-label representations. Meanwhile, *adapted methods* adapt existing binary classification approaches to tag items with multiple labels without requiring problem transformations.

3.1. Classical baseline approach

The baseline approach, which is called the binary relevance (BR) method [20], transforms a multi-label problem into one binary classifier for each label. Hence BR independently trains $\|L\|$ binary classifiers $C_1, \ldots, C_{\|L\|}$. Each classifier C_i is responsible for predicting the 0/1 association for each corresponding label $l_i \in L$. This approach is popular because of its conceptual simplicity, but this method ignores label correlations. Due to this information loss, BR's predicted label sets are likely to contain either too many or too few labels or labels that would never co-occur in practice [20].

For the baseline approach, we use simple TF-IDF (short for Term Frequency – Inverse Document Frequency) features to represent the instances. Basically, every word in the vocabulary set is considered as a feature. Each function is represented as a bag-of-word vector. In this vector, the value of a feature (word) increases proportionally to its count in the function, but it is inversely proportional to the frequency of the word in the corpus. The reason for the selection of TF-IDF for the baseline approach is that TF-IDF has been applied as the baseline representation and shown its potential in various software engineering tasks [25,26] such as code authorship identification [33] or defect prediction [34]. Indeed, the existing studies have empirically shown that the use of TF-IDF could outperform the methods using more sophisticated approaches for certain tasks [25,26], such as the task of Code Authorship Identification.

In this work, we construct the feature set covering unigrams and bigrams. In order to drop some irrelevant features and reduce the dimensionality problem, we apply the Chi-Square test for feature selection to select the most statistically relevant TF-IDF features, i.e., keep only the features exceeding a certain p-value. The reduced vectors are fed to a Binary Relevance Multi-label Classifier with a Gaussian Naive Bayes-based Classifier (Fig. 4).

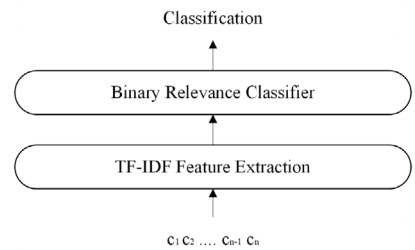

Fig. 4. Traditional Baseline Approach (Base) for vulnerability type identification.

3.2. Deep learning approaches

Neural networks (NN) can be directly adapted to support multi-label classification by simply specifying the number of target labels as the number of nodes in the output layer. For example, a task that has three output labels (classes) will require a NN output layer with three nodes in the output layer. Additionally, till now, the community of SE researchers has paid tremendous efforts to develop powerful code representations for SE classification tasks, such as code authorship identification, code clone detection, source code classification, and software defect prediction. In this work, we investigate the performance of multi-label classification models for VTI adapted from NN approaches using advanced code representations.

With the rapid development of deep learning in SE applications, various code representation techniques have been proposed, which can be categorized into two broad categories: *Non-contextual Embeddings* and *Contextual Embeddings*. Non-contextual embeddings such as Word2vec [28], GloVe [35], fastText [36], Code2vec [37], produce *fixed* representations for words in the vocabulary without considering the meanings of words/code tokens in different contexts. Meanwhile, by contextual embeddings such as CodeBERT [29] and CuBERT [38], the representations of tokens are *adjusted* based on different contexts.

For *Non-contextual Embeddings*, Word2vec has become one of the most popular code embedding techniques for software engineering tasks [24,25] due to its high efficiency. Word2vec produces a low-dimensional semantic space by using two different model architectures: Skip-gram (i.e., starting from a single word to predict its context) or Continuous Bag-of-Words (i.e., starting from the context to predict a word). In this work, we use Word2vec with Skip-gram as a representative non-contextual embedding technique.

For *Contextual Embeddings*, we select CodeBERT [29] as this model has achieved strong improvements on many SE tasks, showing their great generalizability [25,27]. CodeBERT shares the same architecture of BERT (Bidirectional Encoder Representation from Transformer) [21], which uses the bidirectional transformer encoder to effectively exploit both the left and right contexts of a target token. Two objectives are designed for BERT-liked models: masked language model and next sentence prediction. In the masked language model, some of the tokens are randomly masked, and the goal is to predict these masked tokens based on their surrounding unmasked context tokens. For next-sentence prediction, the goal is to predict whether a sentence is the next sentence of the current one to capture the relationships between sentences.

To the best of our knowledge, no vulnerability type identification study has been published. Note that μVulDeePecker [17] could output vulnerable functions with vulnerability type. However, that approach's goal is to determine if a function is clean or of exactly one vulnerability,

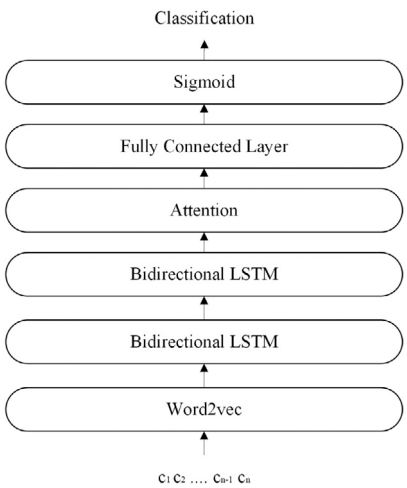

Fig. 5. Vulnerability type identification with Word2vec.

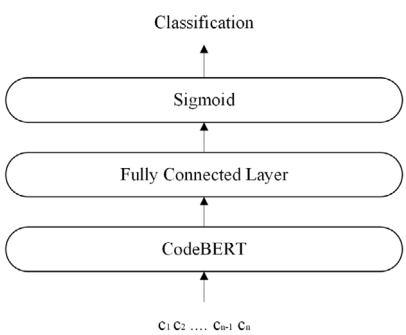

Fig. 6. Vulnerability type identification with CodeBERT.

this is fundamentally different from the task of vulnerability type identification. The detailed differences between VTI approaches and μVulDeePecker are discussed in Section 6.

In this work, for both deep learning models, we follow the typical architectures of classifiers for the general multi-label text classification task instead of designing a complex model. We evaluate the performance of the multi-label classification model using Word2vec proposed by [19]. They use Word2vec to construct a word embedding layer followed by two Bi-LSTM layers, an attention layer, a fully connected, and the sigmoid activation function (Fig. 5). Fig. 6 shows the classification model with CodeBERT proposed by [27]. In this model, CodeBERT is used as an embedding layer encoding every vulnerable function to a vector. The model also feeds the embedding to a fully connected layer and either the sigmoid function to compute the classification. For both models, we use binary cross entropy [19] to compute the loss between the classification and the ground truth.

3.3. Experimental results

Table 1 (*Original Performance* tab) shows the performance of the studied VTI approaches. Note that all our experiments were conducted on a workstation with a P100 GPU, dual vCPUs, and 32 GB RAM.

As seen in *Original Performance* tab of Table 1, VTI with CodeBERT achieved the best performance among the three approaches. However, the performance of this deep pre-trained model is slightly better than that of Base, about 4%-7.5% in F1-scores for *macro average*, *micro average*, *weight average*, and *sample average*. Meanwhile, this advanced method improves 5.5%, 6.4%, and only 1.1% in *exact match ratio*, *hamming score*, and *accuracy*, respectively. Base even significantly outperforms the VTI approach with Word2vec in all the considering metrics. Especially, the micro-average of Base doubles the corresponding figure of Word2vec.

Table 2 shows the detailed classification performance (F1-score) of the three approaches for each vulnerability type. Base achieves F1-scores comparable with those of the CodeBERT-based approach, even slightly better for *Memory Corruption* and *Others*. Additionally, Base significantly improves the performance of the Word2vec-based method for all types. These results empirically demonstrate that Base *with a traditional multi-label classification model and the old-fashioned TF-IDF could achieve a very competitive performance compared to the advanced VTI approaches*.

Especially, Base is much more efficient than the other approaches. Specially, Base is trained **20X** and **40X faster** than the VTI approaches based on Word2vec and CodeBERT. Moreover, Base is also much faster in predicting vulnerability type of vulnerable functions, 3X and 5X faster compared to the Word2vec-based and CodeBERT-based models. Notably, unlike the neural network approaches, Base can be trained and tested **without GPU**, while the Word2vec-based and CodeBERT-based models require GPU and consume much computational resource. This advantage enables a flexible deployment for Base on developers' machines which might not be very powerful with limited resources while retaining a competitive VIT performance.

4. A light-weight method to improve vulnerability type identification

As shown in Section 3.3, Base is *very efficient* because of the adoption of the simple code representation and classification model. However, the appearance of code tokens in all the parts of vulnerable functions is considered equally important. Meanwhile, we observe that there are certain code syntactic elements, such as *function calls*, *assignments*, or *control structures*, which could be more crucial than the others in determining vulnerability types. Thus, instead of considering code tokens in all code syntactic elements equally important, focusing on those crucial code syntactic elements in vulnerable functions could improve the accuracy of VTI predictions. We propose an independent prediction-refining component to enhance the VIT performance of Base while retaining its overall efficiency.

4.1. Design

Definition 1 (***Syntactic Code Element***). A syntactic code element is a syntactical part of programs defined by the programming language in use.

In this work, we use Joern [39], which is widely applied in the existing studies [2,4–7], to analyze vulnerable functions and extract their syntactic code elements. In BigVul, about 95% of vulnerable statements are/contain either *function calls* (78%), *assignments* (44%), *control structures* (38%), or *return statements* (19%). Instead of considering all code elements equally important in VTI, focusing on those crucial kinds of elements in vulnerable functions could improve VTI performance. Based on the observation, we design a lightweight technique that can be applied as an independent step to improve Base's performance. Our idea is that the code tokens, which can be used to distinguish each vulnerability type from the others (so-called, *distinguishing tokens*), are identified by statistically analyzing the training set on the selective syntactic code elements which are critical for VTI such as *function calls*, *assignments*, *control structures*, and *return statements*. For example, *Buffer Overflow* vulnerabilities usually cause by the *assignments* (e.g., assigning too large index) or *function calls* (e.g., copying data larger a buffer's capacity). These distinguishing tokens are used to refine the predictions produced by Base. Particularly, a function f should or should not be of a type t if f has the distinguishing tokens of t.

Table 1
Vulnerability type identification performance.

		Original Performance			*After Enhancement*		
		Base	Word2vec	CodeBERT	Base	Word2vec	CodeBERT
micro-avg	Prec.	0.71	0.55	0.73	0.72	0.60	0.74
	Rec.	0.69	0.43	0.73	0.78	0.56	0.79
	F1	0.70	0.48	0.73	0.75	0.58	0.77
macro-avg	Prec.	0.65	0.41	0.67	0.69	0.70	0.70
	Rec.	0.60	0.24	0.65	0.75	0.48	0.76
	F1	0.62	0.28	0.65	0.71	0.54	0.72
weighted-avg	Prec.	0.71	0.50	0.73	0.73	0.61	0.74
	Rec.	0.69	0.43	0.73	0.78	0.56	0.79
	F1	0.70	0.47	0.73	0.75	0.55	0.76
sample-avg	Prec.	0.65	0.47	0.71	0.73	0.57	0.75
	Rec.	0.68	0.42	0.72	0.80	0.60	0.81
	F1	0.65	0.42	0.70	0.74	0.55	0.76
ExactMatchRatio		0.54	0.30	0.57	0.60	0.32	0.57
HammingScore		0.62	0.39	0.66	0.69	0.49	0.71
Accuracy		0.89	0.84	0.90	0.91	0.86	0.91
Training time (ms)		148,824	2,610,032	4,938,020	148,851	2,610,035	4,938,035
Predict. time (ms)		4,320	12,349	22,282	5,243	13,841	24,028
GPU required		No	Yes	Yes	No	Yes	Yes

Table 2
Performance in F1 score by types.

Types	Base	Word2vec	CodeBERT
Information Gain	0.41	0.00	**0.47**
Privilege Gain	0.63	0.13	**0.67**
Bypass	0.42	0.00	**0.48**
DoS	0.81	0.68	**0.83**
Execution Code	0.64	0.34	**0.66**
Memory Corruption	**0.70**	0.36	0.68
Overflow	0.67	0.37	**0.75**
Others	**0.69**	0.34	0.68

Definition 2 (***Distinguishing Token***). For a type $t \in T$, a distinguishing token in a syntactic code element is a code token which is more/less *prevalent* in the syntactic code elements of the functions having vulnerabilities of type t than in any other types.

Among D_t, which is the collection of cases of type t, the prevalence of a token c regarding a syntactic code element e, ($prev(c, e, D_t)$), is reflected via the ratio of the syntactic code elements e of cases containing c in D_t. Formally, $prev(c, e, D_t) = \frac{count(c,e,D_t)}{\|D_t\|}$, where $count(c, e, D_t)$ is the number of cases where the elements e contain c. Token c is a *positive distinguishing tokens* of t, $c \in \Delta^+(t, e)$, if c is more prevalent in the cases of type t than any other types:

$$dis^+(c, e, t, D) = \frac{prev(c, e, D_t)}{\max_{t' \in T \setminus \{t\}} prev(c, e, D_{t'})} > 1$$

When $\max_{t' \in T \setminus \{t\}} prev(c, e, D_{t'}) = 0$, then $dis^+(c, e, t, D)$ is infinity. In that case, c appears in D_t only, not in the others.

Meanwhile, regarding a syntactic code element e, there are certain tokens which are more prevalent in the cases of the other types rather than t. Such code tokens are considered as *negative distinguishing tokens* of t regarding e, $c \in \Delta^-(t, e)$. Formally, $c \in \Delta^-(t, e)$ if c satisfies the following condition:

$$dis^-(c, e, t, D) = \frac{\min_{t' \in T \setminus \{t\}} prev(c, e, D_{t'})}{prev(c, e, D_t)} > 1$$

If c is never in any case of type t, then $prev(c, e, D_t) = 0$ and $dis^-(c, e, t, D) = \infty$. Intuitively, regarding a syntactic code element, *when a case has a positive distinguishing token of type t, then the vulnerable function is likely to have a vulnerability of t* ($Rule^+$). Similarly, *if the case has a negative distinguishing token of type t, the vulnerable function is likely to be a case of any types other than t* ($Rule^-$).

A vulnerable function f might satisfy neither $Rule^+$nor $Rule^-$. The prediction for t has to rely on the prediction of Base. Thus, we apply $Rule^+$and $Rule^-$to design a technique which can be used as ***an independent component*** combined with the VTI model. Particularly, for the prediction Z of f produced by Base, we apply $Rule^+$and $Rule^-$to refine Z to produce the final prediction Z'. Particularly, for a type $t \in T$, $Z'[t]$, which is the syntactic code element corresponding t in Z', is refined based on that of Z (i.e., $Z[t]$):

- If a syntactic code element e of f contains $c \in \Delta^+(t, e)$, yet $Z[t] = 0$, then $Z'[t] = 1$.
- If a syntactic code element e' of f contains $c' \in \Delta^-(t, e)$, yet $Z[t] = 1$, then $Z'[t] = 0$.
- Otherwise, $Z'[t] = Z[t]$.

In fact, applying the observation, that certain syntactic code elements are more important than others in VTI, as a lightweight independent component could expand the applicability of our technique. Indeed, the refining step using our technique could be applied as an independent component of any VTI approach. We will show the performance of the other approaches when combined with our technique in Section 4.2.

4.2. VIT performance improvement

Improving* Base*'s VTI performance. To evaluate the effectiveness of our method in improving VIT performance, we apply our method as the post-processing step of Base (so-called Enhanced Base). Note that all our experiments were conducted on a workstation with a P100 GPU, dual vCPUs, and 32 GB RAM.

After enhancement, 308 predictions in 1,055 cases are affected with an accuracy rate of 84%. In other words, there are 260/308 predictions are accurately corrected. Table 1 (tab *After Enhancement*) shows the VIT performance of Base with the post-processing step. Compared to Base (tab *Original Performance*), after applying the post-processing step, the macro-average precision increases by 6.1%, while the macro-average recall is significantly improved by 25%. This means that for a vulnerability type t, Enhanced Base not only identified much more the cases of t (higher recall) but also is more precise in identifying t (higher precision). Indeed, as shown in Table 3, the precision of 5/8 types and recall of all types are improved. Especially, the precision for *Information Gain* is improved by 37%, while Enhanced Base doubles the recall of Base for this type.

For the micro-average in Table 1, the precision and recall are slightly improved by 1.4% and 13.0%. The reason is that for certain types, such as *DoS* or *Overflow*, with more cases than the other types,

Table 3
VTI performance of Base and Enhanced Base by types.

Types	Base			Enhanced Base		
	Precision	*Recall*	*F1*	*Precision*	*Recall*	*F1*
Information Gain	0.41	0.42	0.41	0.56	0.83	0.67
Privilege Gain	0.75	0.55	0.63	0.80	0.74	0.77
Bypass	0.43	0.42	0.42	0.55	0.75	0.64
DoS	0.81	0.80	0.81	0.81	0.89	0.84
Execution Code	0.68	0.61	0.64	0.70	0.66	0.68
Memory Corruption	0.82	0.62	0.70	0.82	0.63	0.71
Overflow	0.65	0.69	0.67	0.66	0.74	0.70
Others	0.66	0.72	0.69	0.66	0.72	0.69

Table 4
The effects of the post-processing step on the VTI approaches.

	Base	Word2vec	CodeBERT
No. of affected predictions	308	428	237
Accuracy rate	0.74	0.80	0.72

Base already learns better in identifying these types. Thus, Enhanced Base did not improve Base much for those types. Consequently, micro-average metrics, which are calculated based on individual TP, FP, and FN, were not improved much. This reason explains the slight improvement in the weighted-average. Meanwhile, for samples-average, the improvements in the precision and recall are more significant, 12% and 17%, respectively. This demonstrates the effectiveness of the Enhanced Base in improving the individual predictions of Base.

The improvements in these above metrics result in the increases of all *exact match ratio*, *hamming score*, and *accuracy* (Table 1). As seen, Enhanced Base can give 3/5 fully correct predictions. Meanwhile, more than 2/3 predicted types (*hamming score* of 69%) are accurately given by Enhanced Base. Compared to Base, both the *exactly match ratio* and *hamming score* of Enhanced Base are more than 10% better. However, the improvement in *accuracy* is only 2.2%. This is because the cases whose vulnerability type set is small are very popular, and models tend to predict very few types. Meanwhile, *accuracy* considers both labels 0 and 1 in prediction vectors and ground-truth vectors. Thus, the accuracy of Base in each case is already high. This leads to the low improvement by Enhanced Base in *accuracy*.

Compared to CodeBERT-based approach. As seen in Table 1, Enhanced Base achieved better performance in all the metrics (except micro-average precision) compared to the CodeBERT-based approach. For micro-average, although the precision of Enhanced Base is slightly lower than that of CodeBERT (0.72 vs. 0.73), the improvement of Enhanced Base in the recall is more significant (0.78 vs. 0.73). This results in an improvement in micro-average F1. Analyzing the cases where CodeBERT-based can perform well while Enhanced Base did not, we found that these cases have quite complex logic and belong to multiple types which have a causal relationship. For example, *DoS* vulnerabilities could be caused by *Overflow* ones. Meanwhile, Enhanced Base does not consider the relationship between types. Thus, although they have certain distinguishing tokens of a type, Enhanced Base might fail to infer that a vulnerable code also has other types.

In our implementation, the preparing (prep.) step to extract distinguishing tokens and the model training step are performed in parallel. Thus, the total time for model training and preparing is still 2.48 min. Meanwhile, the predicting time slightly increases from 4.0 to 5.2 s. These time costs are much more efficient than those of CodeBERT.

Overall, we can conclude that *a simple model combined with a lightweight component could achieve better VTI performance and be much more efficient than advanced deep pre-trained approaches*. This could be very meaningful for users who want to achieve both high effectiveness and efficiency in VTI.

Effectiveness in improving approaches. Inspired by the success of our method in improving the performance of Base, we apply the technique to enhance the other approaches. Table 1 (tab *After Enhancement*) shows the VTI performance of Word2vec and CodeBERT approaches after applying our method as a post-processing step. Compared to their performance before enhancement (Table 1), the performance of all these approaches is improved. The improvements for CodeBERT are minor but still visible by up to 10% in macro-average F1. The numbers of affected predictions and accuracy rates for Word2vec and CodeBERT are shown in Table 4. As seen, the effect of the post-processing step on the Word2vec-based approach is more significant, with a higher accuracy rate compared to that of the CodeBERT-based method. The reason could be that the advanced deep pre-trained CodeBERT can capture well certain degrees of our rules. Thus, our method was not very effective in improving the CodeBERT-based VIT approach. However, the improvements in all the micro-average, macro-average, weighted-average, and sample-average metrics for Word2vec are significant. Among these metrics, macro-average F1 increases by 92.8% after applying our post-processing step. These results demonstrate that *our approach is very effective in improving the VTI performance of not only* Base*, but also the others.*

5. Threats to validity

The main threats to the validity of our work consist of internal, construct, and external threats.

Threats to internal validity include the influence of the method used to identify the code elements (e.g., function calls, assignments, or control structures). To reduce this threat, we use Joern [39] code analyzer, which is widely used in existing studies [2,4,5,7].

Threats to construct validity relate to the suitability of our evaluation procedure. We used *exact match ratio*, *hamming score*, *accuracy*, *macro-average*, *micro-average*, *weighted-average*, and *sample-average*. They are the classical evaluation measures for multi-label classification [18].

Threats to external validity mainly lie in the selection of multi-label classification models used in our experiments. To mitigate this threat, we select the representative models which are well-known for NLP and SE tasks. Base is considered as the baseline approach for general multi-label text classification, while all Word2vec, Glove, and CodeBERT are reputed and shown to be effective in many SE tasks. The dataset used in our experiments might not be representative or not very high-quality. To reduce this threat, we used the largest public dataset [13], which is collected from a large number of real-world projects and widely used in existing vulnerability detection studies [2–4]. Additionally, our data has only C/C++ code. Thus, we cannot claim that similar results would have been observed in other programming languages. Further studies are needed to validate and generalize our findings to other languages.

6. Related work

Vulnerability/Bug Detection and Prevention. Various methods have been proposed to determine if a code component (component, file, function/method, or statement/line) is vulnerable. The rule-based techniques apply static analyzers and leverage seen vulnerability patterns, such as FlawFinder [40] or Coverity [41]. Recently, several deep-learning based approaches have been introduced [5,8–11,42–44]. VulDeePecker [5] and SySeVR [6] introduce tools to detect *slice-level* vulnerabilities, which are more fine-grained. IVDetect [4], which is a graph-based neural network model, is proposed to detect vulnerabilities at the function level and use a model interpreter to identify vulnerable statements in the detected suspicious functions. LineVul [3] and LineVD [2] apply CodeBERT in their own way and have been shown that they are more effective than IVDetect in detecting vulnerable functions and lines/statements. VelVet [7] builds graph-based models to detect vulnerable statements. Our work could complement well with the existing automated vulnerability detection approaches. Particularly, the type identification step could be applied after developers use a vulnerability detection method to quickly interpret and fix the vulnerable

functions. Our work could also be applied to determine the vulnerability types of a vulnerable code component before applying one or more vulnerability detection approaches specialized for certain vulnerability types, such as the approach to memory-related vulnerabilities [43].

Our work might related VUDENC [45] Wartschinski et al. and μVulDeePecker [17] by Zou et al. which focus on vulnerability detection and can implicitly indicate the type of vulnerability. In other words, given a piece of code, these techniques determine if the code is benign or belongs to one/some vulnerability types. VUDENC [45] uses separate LSTM binary classification models to determine if a piece of code is neutral/benign or belongs to some vulnerability types. In this paper, we focus on the specific task of vulnerability type identification integrated after vulnerability detection to provide developers with the type of vulnerabilities effectively identified by the existing vulnerability detection techniques [2–6] which determine if the given code is benign or vulnerable. Meanwhile, the goal of μVulDeePecker [17] is to determine if a function is clean (not vulnerable) or of exactly one vulnerability. However, our work differs from μVulDeePecker in three fundamental aspects. First, μVulDeePecker is a vulnerability detection approach which is designed to decide if a function is vulnerable. In other words, the input of that approach is a function that has not known whether it is vulnerable or not. Meanwhile, our work is designed to apply as a step after detecting vulnerable functions. Therefore, the input function of our work is assumed to be vulnerable. Moreover, μVulDeePecker assumes that a vulnerable function has only one type. Thus, the multi-class classification is considered in μVulDeePecker. Meanwhile, a function could belong to multiple types. Hence, in our work, the VTI problem is modeled as the multi-label classification task. Finally, μVulDeePecker is designed for the function call vulnerabilities, while our work has no limit to the kind of vulnerabilities.

Learning-based approaches for SE tasks. Several studies have been proposed for specific SE tasks, including code suggestion/completion [46–50], program synthesis [51], pull request description generation [52,53], code summarization [54–56], code clones [57], fuzz testing [58], code-text translation [59], and program repair [60,61]. Recently, several learning techniques have been proposed to learn representing source code for specific SE applications [37,62] or general SE tasks [29,38,63].

7. Conclusion

In this work, we investigate the problem of vulnerability type identification (VTI) after vulnerability detection. The problem is modeled as the multi-label classification task. Particularly, each detected vulnerable function is considered as a document, and the set of vulnerability types of the function could be considered as the label set of the corresponding document. This NLP task has been effectively addressed by pre-training, then fine-tuning the framework with deep pre-trained embedding models. The existing studies show that the deep pre-trained embedding models specialized for code have also shown their effectiveness for many classification tasks in software engineering. In this paper, we experimentally evaluate the performance of the well-known and advanced pre-trained models for VTI on a large set of vulnerabilities in various types. Surprisingly, their performance is not much better than the VTI performance of the traditional baseline classification model with an old-fashioned bag-of-word TF-IDF. Meanwhile, these neural network approaches cost much more time and require GPU. We also introduce a lightweight independent component to enhance the predictions of the baseline approach. Our idea is that the types of vulnerabilities could strongly correlate to certain code tokens (distinguishing tokens) in several crucial parts of programs. The distinguishing tokens for a vulnerability type are statistically identified based on their prevalence in the type versus the others. Our results show that the baseline approach enhanced by our component can outperform the state-of-the-art deep pre-trained methods while retaining very high efficiency. Furthermore, the proposed technique could also improve the neural network approaches by up to 92.8% in macro-average F1.

CRediT authorship contribution statement

Hieu Dinh Vo: Conceptualization, Methodology, Formal analysis, Software, Investigation, Data curation, Writing – review & editing. **Son Nguyen:** Conceptualization, Methodology, Formal analysis, Validation, Investigation, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Son Nguyen reports financial support was provided by VNU University of Engineering and Technology.

Data availability

Data will be made available on request.

Acknowledgments

This work has been supported by VNU University of Engineering and Technology under project number CN22.14.

References

[1] H. Krasner, The cost of poor software quality in the US: A 2020 report, Proc. Consortium Inf. Softw. QualityTM (CISQTM) (2021).
[2] D. Hin, A. Kan, H. Chen, M.A. Babar, LineVD: Statement-level vulnerability detection using graph neural networks, in: IEEE/ACM 19th International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022, IEEE, 2022, pp. 596–607.
[3] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories, MSR, IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 608–620.
[4] Y. Li, S. Wang, T.N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 292–303.
[5] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: 25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018, The Internet Society, 2018.
[6] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secure Comput. 19 (04) (2022) 2244–2258.
[7] Y. Ding, S. Suneja, Y. Zheng, J. Laredo, A. Morari, G. Kaiser, B. Ray, VELVET: A noVel ensemble learning approach to automatically locate VulnErable statements, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2022, pp. 959–970.
[8] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, POSTER: Vulnerability discovery with function representation learning from unlabeled projects, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2539–2541.
[9] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, Y. Wu, VulSniper: Focus your attention to shoot fine-grained vulnerabilities, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, 2019, pp. 4665–4671.
[10] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Proceedings of the 33rd International Conference on Neural Information Processing Systems, 2019, pp. 10197–10207.
[11] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (09) (2022) 3280–3296.
[12] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 519–531.
[13] J. Fan, Y. Li, S. Wang, T.N. Nguyen, AC/C++ code vulnerability dataset with code changes and CVE summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories, 2020, pp. 508–512.
[14] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, F. Palomba, The secret life of software vulnerabilities: A large-scale empirical study, IEEE Trans. Softw. Eng. 49 (1) (2022) 44–63.

[15] H. Pearce, B. Tan, B. Ahmad, R. Karri, B. Dolan-Gavitt, Can openAI codex and other large language models help us fix security bugs? 2021, arXiv preprint arXiv:2112.02125.

[16] Z. Chen, S. Kommrusch, M. Monperrus, Neural transfer learning for repairing security vulnerabilities in c code, IEEE Trans. Softw. Eng. 49 (1) (2022) 147–165.

[17] D. Zou, S. Wang, S. Xu, Z. Li, H. Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, IEEE Trans. Dependable Secure Comput. 18 (5) (2019) 2224–2236.

[18] M.-L. Zhang, Z.-H. Zhou, Multilabel neural networks with applications to functional genomics and text categorization, IEEE Trans. Knowl. Data Eng. 18 (10) (2006) 1338–1351.

[19] I. Ameer, N. Bölücü, M.H.F. Siddiqui, B. Can, G. Sidorov, A. Gelbukh, Multi-label emotion classification in texts using transfer learning, Expert Syst. Appl. 213 (2023) 118534.

[20] S. Godbole, S. Sarawagi, Discriminative methods for multi-labeled classification, in: Pacific-Asia Conference on Knowledge Discovery and Data Mining, Springer, 2004, pp. 22–30.

[21] J.D.M.-W.C. Kenton, L.K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NAACL-HLT, 2019, pp. 4171–4186.

[22] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R.R. Salakhutdinov, Q.V. Le, XLNet: Generalized autoregressive pretraining for language understanding, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. dÁlché Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 32, Curran Associates, Inc., 2019, pp. 5753–5763.

[23] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, M. Auli, Fairseq: A fast, extensible toolkit for sequence modeling, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations), 2019, pp. 48–53.

[24] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, X. Liu, A novel neural source code representation based on abstract syntax tree, in: 2019 IEEE/ACM 41st International Conference on Software Engineering, ICSE, IEEE, 2019, pp. 783–794.

[25] Z. Ding, H. Li, W. Shang, T.-H.P. Chen, Can pre-trained code embeddings improve model performance? Revisiting the use of code embeddings in software engineering tasks, Empir. Softw. Eng. 27 (3) (2022) 1–38.

[26] H.J. Kang, T.F. Bissyandé, D. Lo, Assessing the generalizability of code2vec token embeddings, in: 2019 34th IEEE/ACM International Conference on Automated Software Engineering, 2019, pp. 1–12.

[27] X. Zhou, D. Han, D. Lo, Assessing generalizability of CodeBERT, in: 2021 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2021, pp. 425–436.

[28] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient estimation of word representations in vector space, in: Y. Bengio, Y. LeCun (Eds.), 1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings, 2013.

[29] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 1536–1547.

[30] S. Elder, N. Zahan, R. Shu, M. Metro, V. Kozarev, T. Menzies, L. Williams, Do I really need all this work to find vulnerabilities? Empir. Softw. Eng. 27 (6) (2022) 1–78.

[31] Vulnerability distribution by types, 2023, https://www.cvedetails.com/vulnerabilities-by-types.php (Accessed: 23 April 2023).

[32] V. Okun, A. Delaitre, P.E. Black, et al., Report on the static analysis tool exposition (sate) iv, NIST Special Publ. 500 (2013) 297.

[33] M. Abuhamad, T. AbuHmed, A. Mohaisen, D. Nyang, Large-scale and language-oblivious code authorship identification, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 101–114.

[34] S. Wang, T. Liu, L. Tan, Automatically learning semantic features for defect prediction, in: 2016 IEEE/ACM 38th International Conference on Software Engineering, ICSE, IEEE, 2016, pp. 297–308.

[35] J. Pennington, R. Socher, C.D. Manning, Glove: Global vectors for word representation, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP, 2014, pp. 1532–1543.

[36] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching word vectors with subword information, Trans. Assoc. Comput. Linguist. 5 (2017) 135–146.

[37] U. Alon, M. Zilberstein, O. Levy, E. Yahav, Code2vec: Learning distributed representations of code, Proc. ACM Programm. Lang. 3 (POPL) (2019) 1–29.

[38] A. Kanade, P. Maniatis, G. Balakrishnan, K. Shi, Learning and evaluating contextual embedding of source code, in: Proceedings of the 37th International Conference on Machine Learning, ICML '20, JMLR.org, 2020.

[39] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: 2014 IEEE Symposium on Security and Privacy, IEEE, 2014, pp. 590–604.

[40] Flawfinder, https://dwheeler.com/flawfinder/.

[41] Coverity, http://scan.coverity.com/.

[42] G. Lin, S. Wen, Q.-L. Han, J. Zhang, Y. Xiang, Software vulnerability detection using deep neural networks: A survey, Proc. IEEE 108 (10) (2020) 1825–1848.

[43] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, C. Tao, MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1456–1468.

[44] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 519–531.

[45] L. Wartschinski, Y. Noller, T. Vogel, T. Kehrer, L. Grunske, VUDENC: Vulnerability detection with deep learning on a natural codebase for Python, Inf. Softw. Technol. 144 (C) (2022).

[46] S. Nguyen, H. Phan, T. Le, T.N. Nguyen, Suggesting natural method names to check name consistencies, in: 2020 IEEE 42nd International Conference on Software Engineering, IEEE, 2020, pp. 1372–1384.

[47] A. Hindle, E.T. Barr, M. Gabel, Z. Su, P. Devanbu, On the naturalness of software, Commun. ACM 59 (5) (2016) 122–131.

[48] S. Nguyen, T. Nguyen, Y. Li, S. Wang, Combining program analysis and statistical language model for code statement completion, in: 2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2019, pp. 710–721.

[49] C.T. Manh, K.T. Trung, T.M. Nguyen, T.-T. Nguyen, S. Nguyen, H.D. Vo, API parameter recommendation based on language model and program analysis, in: 2021 28th Asia-Pacific Software Engineering Conference, APSEC, IEEE, 2021, pp. 492–496.

[50] S. Nguyen, C.T. Manh, K.T. Tran, T.M. Nguyen, T.-T. Nguyen, K.-T. Ngo, H.D. Vo, ARist: An effective API argument recommendation approach, J. Syst. Softw. (2023) 111786.

[51] T. Gvero, V. Kuncak, Synthesizing Java expressions from free-form queries, in: Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, 2015, pp. 416–432.

[52] X. Hu, G. Li, X. Xia, D. Lo, Z. Jin, Deep code comment generation, in: 2018 IEEE/ACM 26th International Conference on Program Comprehension, ICPC, IEEE, 2018, pp. 200–20010.

[53] Z. Liu, X. Xia, C. Treude, D. Lo, S. Li, Automatic generation of pull request descriptions, in: 34th IEEE/ACM International Conference on Automated Software Engineering, IEEE, 2019, pp. 176–188.

[54] S. Iyer, I. Konstas, A. Cheung, L. Zettlemoyer, Summarizing source code using a neural attention model, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2016, pp. 2073–2083.

[55] A. Mastropaolo, S. Scalabrino, N. Cooper, D.N. Palacio, D. Poshyvanyk, R. Oliveto, G. Bavota, Studying the usage of text-to-text transfer transformer to support code-related tasks, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering, ICSE, IEEE, 2021, pp. 336–347.

[56] Y. Wan, Z. Zhao, M. Yang, G. Xu, H. Ying, J. Wu, P.S. Yu, Improving automatic source code summarization via deep reinforcement learning, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, 2018, pp. 397–407.

[57] L. Li, H. Feng, W. Zhuang, N. Meng, B. Ryder, Cclearner: A deep learning-based clone detection approach, in: International Conference on Software Maintenance and Evolution, IEEE, 2017, pp. 249–260.

[58] P. Godefroid, H. Peleg, R. Singh, Learn&fuzz: Machine learning for input fuzzing, in: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2017, pp. 50–59.

[59] H.A. Nguyen, H.D. Phan, S.S. Khairunnesa, S. Nguyen, A. Yadavally, S. Wang, H. Rajan, T. Nguyen, A hybrid approach for inference between behavioral exception API documentation and implementations, and its applications, in: 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13.

[60] N. Jiang, T. Lutellier, L. Tan, CURE: Code-aware neural machine translation for automatic program repair, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering, ICSE, IEEE, 2021, pp. 1161–1173.

[61] Y. Ding, B. Ray, P. Devanbu, V.J. Hellendoorn, Patching as translation: The data and the metaphor, in: 2020 35th IEEE/ACM International Conference on Automated Software Engineering, IEEE, 2020, pp. 275–286.

[62] Y. Li, S. Wang, T.N. Nguyen, S. Van Nguyen, Improving bug detection via context-based code representation learning and attention-based neural networks, Proc. ACM Programm. Lang. 3 (OOPSLA) (2019) 1–30.

[63] N.D. Bui, Y. Yu, L. Jiang, Infercode: Self-supervised learning of code representations by predicting subtrees, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering, ICSE, IEEE, 2021, pp. 1186–1197.